

Plugin interface rename and mapping

Every registered interface in GeoIPS, the name it carries today, and the name it carries under the v2.0.0 *Plugin* / *PluginConfig* split.

Nothing here is committed — this worksheet exists so the naming can be argued with before it lands in the Specification.

Plugin What is currently called a class-based plugin. Python, implements behavior.	PluginConfig What is currently called a YAML-based plugin. Declarative, configures a Plugin.
--	--

Every PluginConfig interface maps to exactly one Plugin interface. Several PluginConfig interfaces may map to the same Plugin. A Plugin interface does not require a PluginConfig.

Kind is the singular form of the registered interface name — the value a workflow step gives to `kind` to select which interface resolves the step. The column below shows the *new* kind; the value it replaces is the singular of the current interface name on the same row. Renaming an interface renames a value that appears in every existing workflow, which is what the deprecation register at the foot of this page exists to absorb.

```
steps:
  - kind: writer           # was: output_formatter
    name: imagery_annotated
```

INTERFACE MAP

- RENAMEname changes
- NEWinterface does not exist yet
- UNCHANGEDname is fine as it is
- DEPRECATEDcompatibility surface only

#	CURRENT INTERFACE	NEW INTERFACE	KIND	→ PLUGIN	→ PLUGINCONFIG	STATUS	NOTES
PLUGINSclass-based today							
1	algorithms	algorithms	algorithm	Algorithms	AlgorithmConfigs	UNCHANGED	Gains a new config interface — row 19.
2	colormappers	colormaps	colormap	Colormaps	ColormapConfigs	RENAME	Gains a new config interface — row 20.
3	coverage_checkers	coverage_checkers	coverage_checker	CoverageCheckers	—	UNCHANGED	
4	databases	reporters	reporter	Reporters	—	RENAME	The interface reports run results outward; the destination happening to be a database is an implementation detail of the plugin.
5	filename_formatters	filename_formatters	filename_formatter	FilenameFormatters	—	UNCHANGED	Possible future FilenameFormatterConfigs.
6	interpolators	interpolators	interpolator	Interpolators	—	UNCHANGED	
7	output_checkers	output_checkers	output_checker	OutputCheckers	—	UNCHANGED	
8	output_formatters	writers	writer	Writers	WriterConfigs	RENAME	Gains a new config interface — row 21.

#	CURRENT INTERFACE	NEW INTERFACE	KIND	→ PLUGIN	→ PLUGINCONFIG	STATUS	NOTES
9	procflows	—	—	—	—	DEPRECATED	Being retired in favour of Order-Based Processing, so it needs no new name at all. procflow is removed as a kind rather than redirected.
10	readers	readers	reader	Readers	—	UNCHANGED	Possible future ReaderConfigs.
11	title_formatters	title_formatters	title_formatter	TitleFormatters	—	UNCHANGED	Possible future TitleFormatterConfigs.
12	validators	validators	validator	Validators	—	UNCHANGED	
PLUGINCONFIGS YAML-based today							
13	feature_annotators	writer_feature_configs	writer_feature_config	Writers	WriterFeatureConfigs	RENAME	Kept as its own interface and referenced by WriterConfigs rather than absorbed into it, so one annotator definition can be shared across writers.
14	gridline_annotators	writer_gridline_configs	writer_gridline_config	Writers	WriterGridlineConfigs	RENAME	Same arrangement as row 13.
15	product_defaults	product_defaults	product_default	Workflows	product_defaults	DEPRECATED	Name retained. Converts to a WorkflowConfig at runtime.
16	products	products	product	Workflows	products	DEPRECATED	Name retained. Converts to a WorkflowConfig at runtime.
17	sectors	sector_configs	sector_config	Sectors	SectorConfigs	RENAME NEW	Name reassignment. sectors currently registers this PluginConfig and would register the new Plugin instead — see the note below. Static sectors only; dynamic sector interfaces are deferred.
18	workflows	workflow_configs	workflow_config	Workflows	WorkflowConfigs	RENAME NEW	Name reassignment , the same shape as row 17. The Plugin becomes an explicitly registered interface rather than the implicit one in interfaces/class_based/workflow.py.
NEW PLUGINCONFIGS no current interface							
19	—	algorithm_configs	algorithm_config	Algorithms	AlgorithmConfigs	NEW	Declarative algorithm definitions, along the lines of the RGB configs in NRLMMD-GEOIPS/geoips#1484 .
20	—	colormap_configs	colormap_config	Colormaps	ColormapConfigs	NEW	
21	—	writer_configs	writer_config	Writers	WriterConfigs	NEW	References WriterFeatureConfigs and WriterGridlineConfigs.

Dynamic sectors are deferred

`sector_adjusters`, `sector_metadata_generators` and `sector_spec_generators` are deliberately absent from the table. Consolidating them into a single `DynamicSectors / DynamicSectorConfigs` pair is recorded as a tabled design topic rather than decided here. They keep their current names until that discussion concludes, and whether the new `Sectors` Plugin relates to them is deferred along with it.

Two names change meaning rather than disappearing

`sectors` and `workflows` register `PluginConfig` interfaces today. Under this map each becomes the name of a *Plugin* interface, while the existing interface moves to `sector_configs` and `workflow_configs`. The old name stays valid and starts meaning something else.

Every other rename in the table can be handled by aliasing the old kind to the new one. These two cannot: a workflow step reading `kind: workflow` is ambiguous between the interface the author meant and the one the name now denotes. The deprecation register needs a deliberate answer for these two rows — which is the reason they are called out here rather than left to the register.

Products and ProductDefaults reach Workflows through conversion

Both are backwards-compatibility surfaces. A `Product` is converted into a `WorkflowConfig` at runtime and executed by the `Workflows` Plugin, so the one-Plugin-per-`PluginConfig` rule is satisfied through that conversion rather than by giving either its own Plugin.

Their names do not change. Renaming a surface that exists only so existing configurations keep working would defeat the reason it exists.

DEPRECATION REQUIREMENT

Current kinds continue to be accepted, and raise a deprecation warning when encountered. Every rename in the table therefore produces a deprecation-register entry carrying an old kind, a new kind, a warning, and a removal schedule. No rename lands without one.